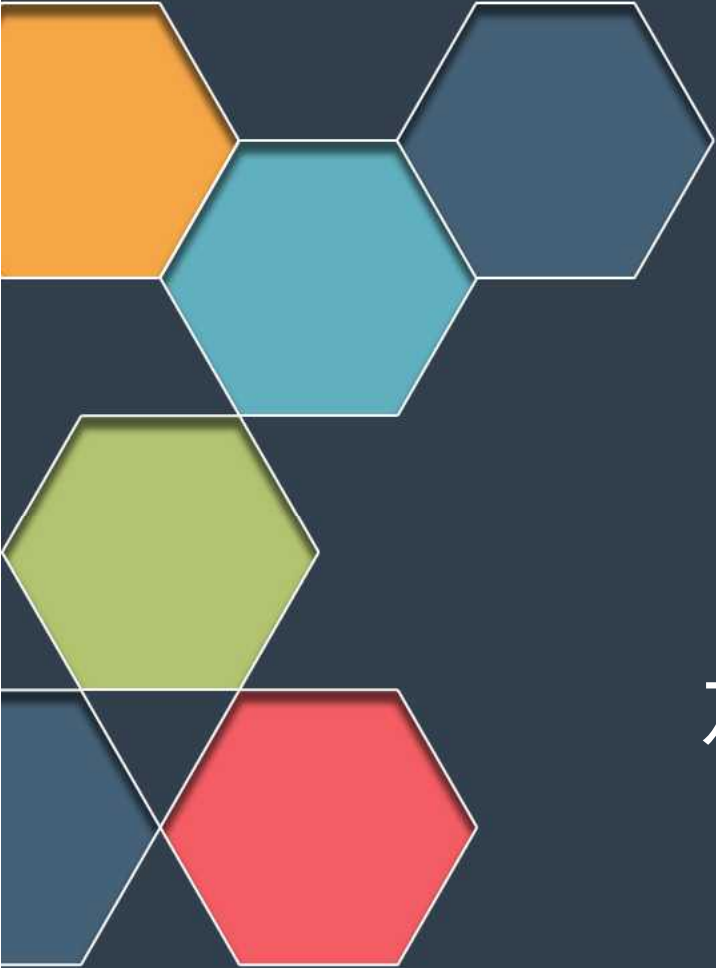


A decorative graphic on the left side of the slide consists of several overlapping hexagons in various colors: orange, teal, dark blue, light green, and red. The hexagons are arranged in a staggered, honeycomb-like pattern.

제12장 신경망(퍼셉트론)

학습 목표

- 신경망에 대하여 이해한다.
- 신경망의 초기모델인 퍼셉트론을 이해한다.
- 퍼셉트론의 작동원리를 이해한다.
- 퍼셉트론 학습 알고리즘을 이해한다.
- 퍼셉트론의 한계점을 인식한다.

01 신경망

- 최근에 많은 인기를 끌고 있는 딥러닝(deep learning)의 시작은 1950년대부터 연구되어 온 인공 신경망(artificial neural network: ANN)이다.
- 인공 신경망은 생물학적인 신경망에서 영감을 받아서 만들어진 컴퓨팅 구조이다.

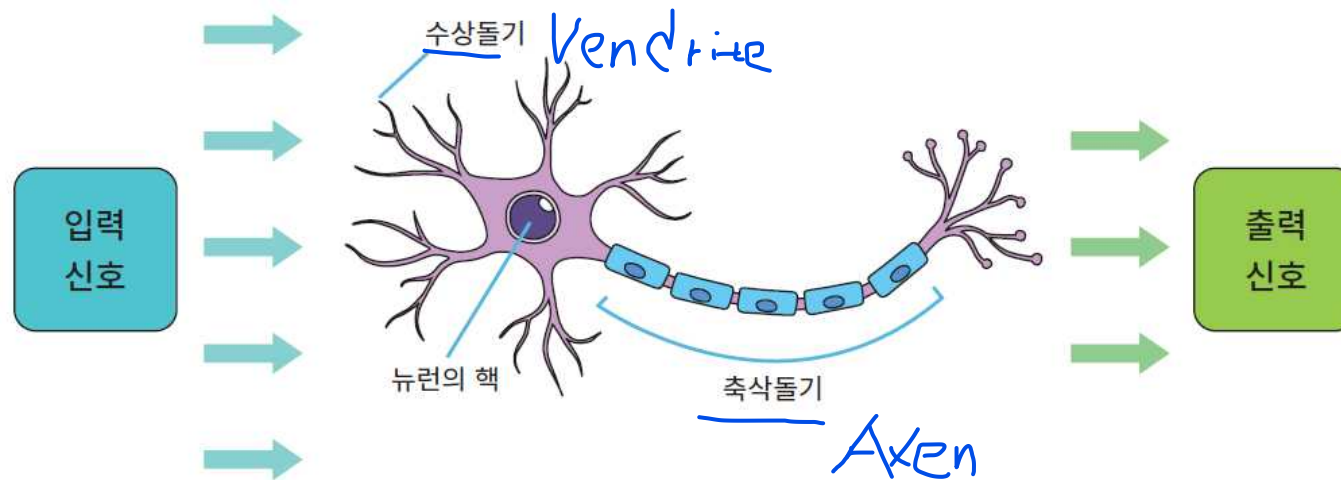


그림 12-1 뉴런의 구조

인공 신경망

- 인공 신경망은 유닛(unit), 또는 노드(node)라고 하는 소자들을 모아
서 만든다. 이들 소자는 생물학적 뉴런을 느슨하게 모델링한 것이다.

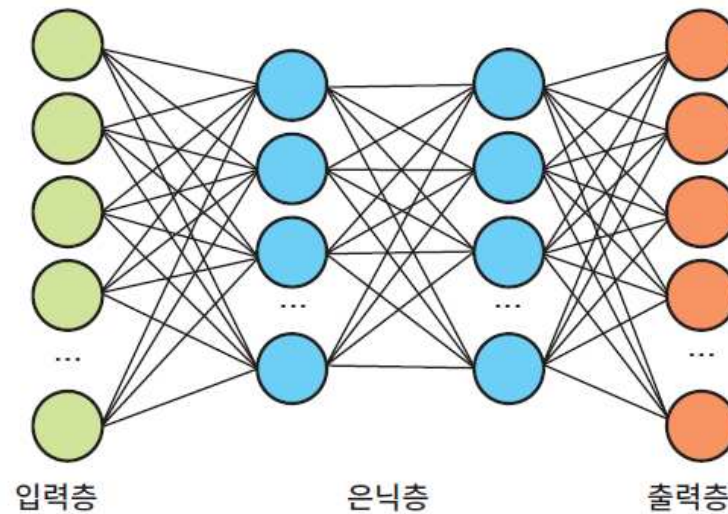
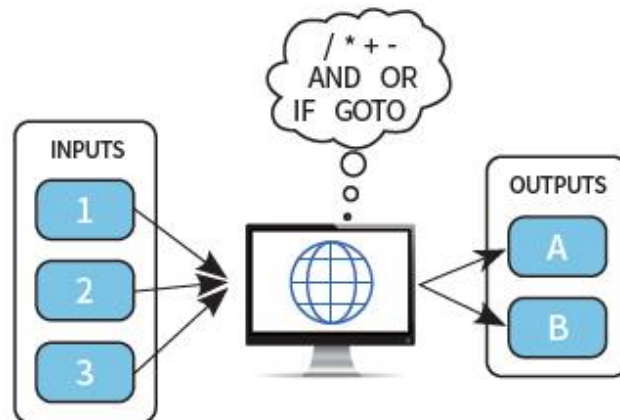


그림 12-2 인공 신경망

전통적인 컴퓨터 vs 인공신경망

	기존의 컴퓨터	인간의 두뇌
처리소자의 개수	10^8 개의 트랜지스터	86×10^{10} 개의 뉴런 860억
처리소자의 속도	<u>10^{12}Hz</u>	<u>10^2Hz</u>
학습기능	없음	있음
계산 스타일	<u>중앙 집중식, 순차적인 처리</u>	<u>분산 병렬 처리</u>

속도는 cpu가 빠르다.

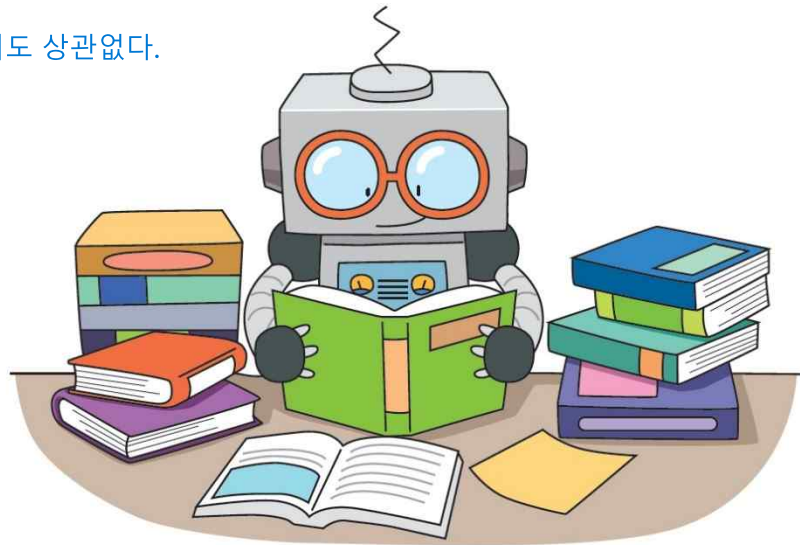


신경망의 장점

- 첫 번째는 **학습이 가능**하다는 점이다. 데이터만 주어진다면 신경망은 예제로부터 배울 수 있다.
 - 신경망이 학습이 가능하다면 이것은 아주 큰 장점이 된다. 신경망은 예제로부터 작업을 수행하는 방법을 스스로 배울 수 있다. 따라서 인간이 프로그램을 작성할 필요가 없다.
- 두 번째는 **몇 개의 소자가 오작동**하더라도 전체적으로는 큰 문제가 발생하지 않는다는 점이다.

우리 두뇌는 신경세포가 몇개 죽어도 상관없다.
그래서 신경망도 똑같다.
학습할 때 일부로 몇 개 꺼놓는다.

Overfitting



신경망의 장점

- 예를 들어 강아지 이미지와 고양이 이미지를 식별하는 작업을 생각해 보자
- 인간은 쉽게 이미지를 인식하지만 인간도 인식의 메커니즘을 정확히 모르기 때문에 인식 알고리즘을 명시적으로 만드는 것은 아주 어려운 일이다.

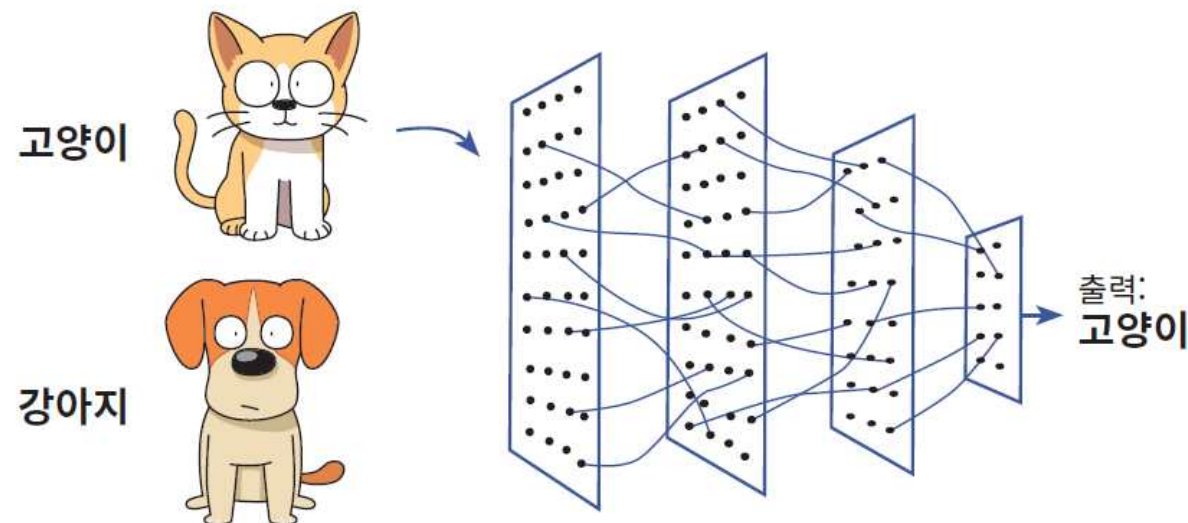


그림 12-4 신경망을 이용한 이미지 인식

뉴런의 구조

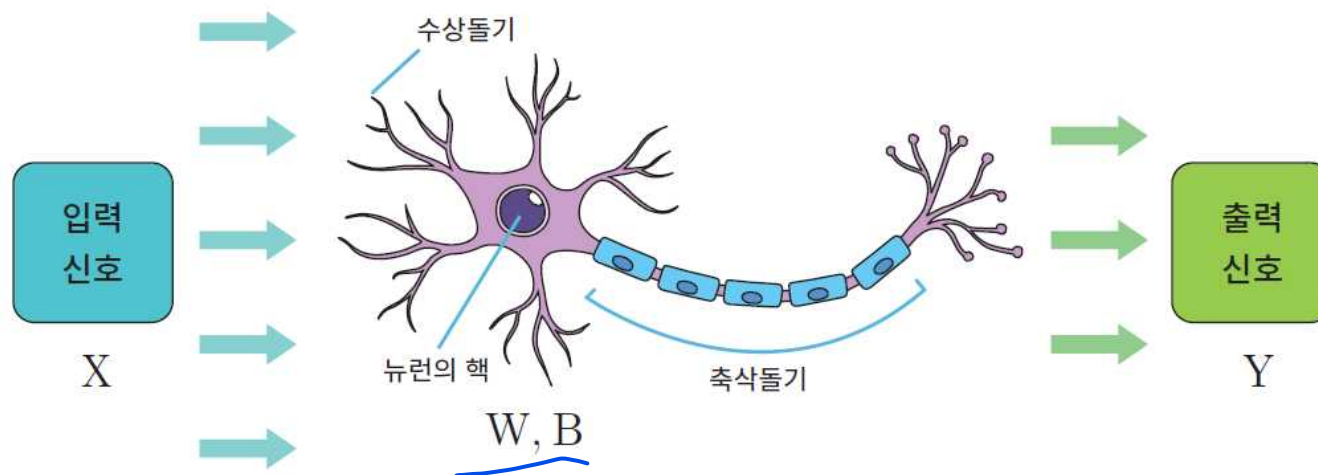


그림 12-5 뉴런의 구조

입력하고 가중치를 곱해서 받는다.

뉴런의 수학적 모델

입력, 가중치 곱한 것 - 임계값

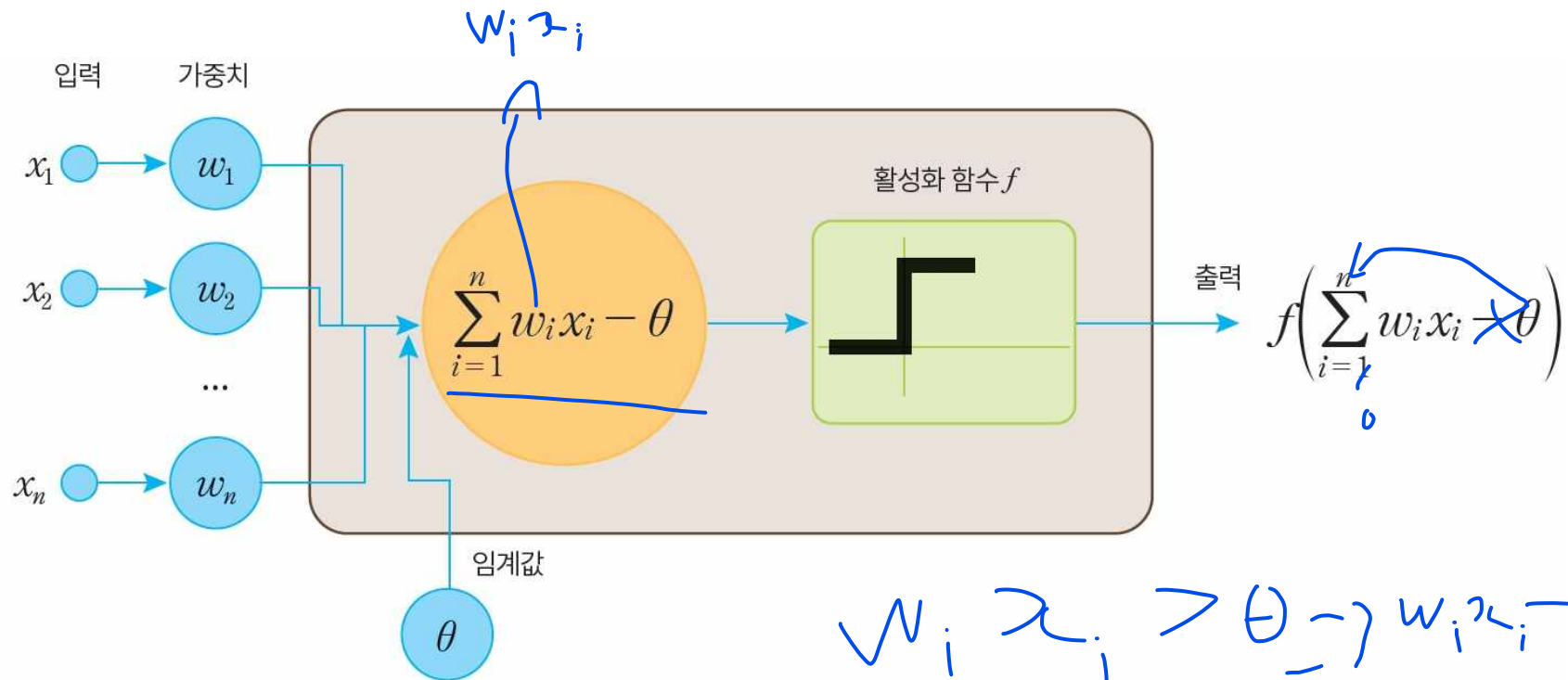


그림 12-5 뉴런의 수학적 구조

새로운 입력의 가중치를 -세타로 정한다..?
x0를 항상 1로 만든다..?

$$\sum_{i=0}^n w_i x_i$$

02 퍼셉트론

- 퍼셉트론(perceptron)은 1957년에 로젠블라트(Frank Rosenblatt)가 고안한 인공 신경망이다.

원래는 학습방법이 없었는데,
이 사람이 1개짜리 학습방법 만들

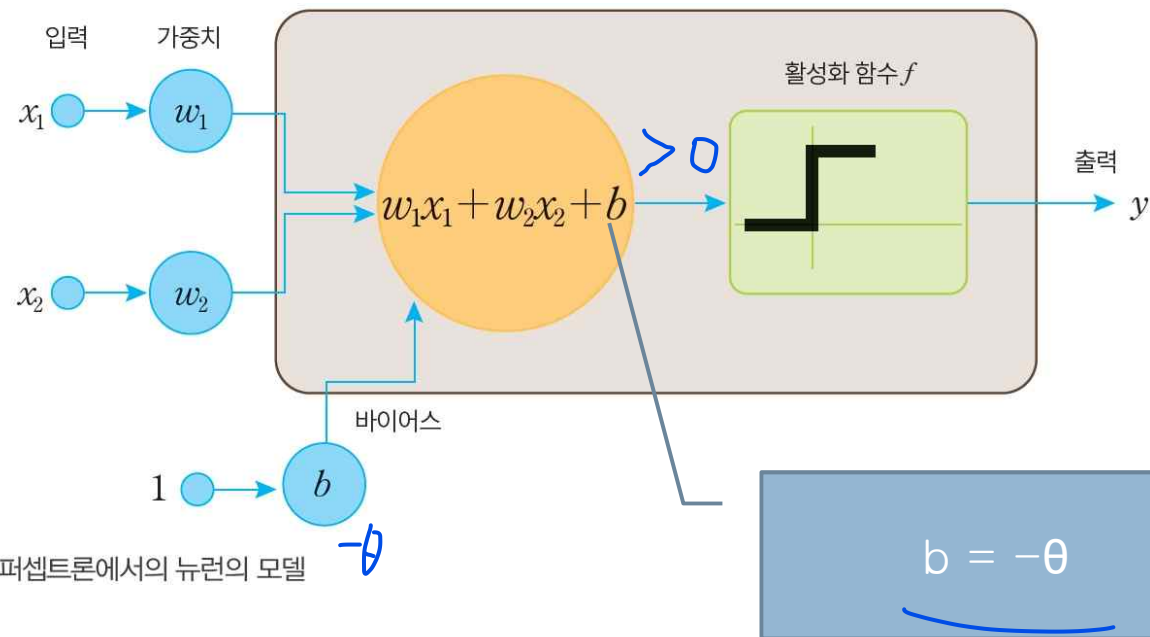


그림 12-6 퍼셉트론에서의 뉴런의 모델

퍼셉트론

- 뉴런에서는 입력 신호의 가중치 합이 어떤 임계값을 넘는 경우에만 뉴런이 활성화되어서 1을 출력한다. 그렇지 않으면 0을 출력한다.

$$y = \begin{cases} 1 & \text{if } (w_1x_1 + w_2x_2 + b \geq 0) \\ 0 & \text{otherwise} \end{cases}$$



NOTE: 바이어스와 임계값

생물학적인 뉴런에서는 입력 총합이 어떤 임계값을 넘어야 뉴런이 활성화 된다고 하여서 뉴런의 수식을 다음과 같이 표기하기도 한다.

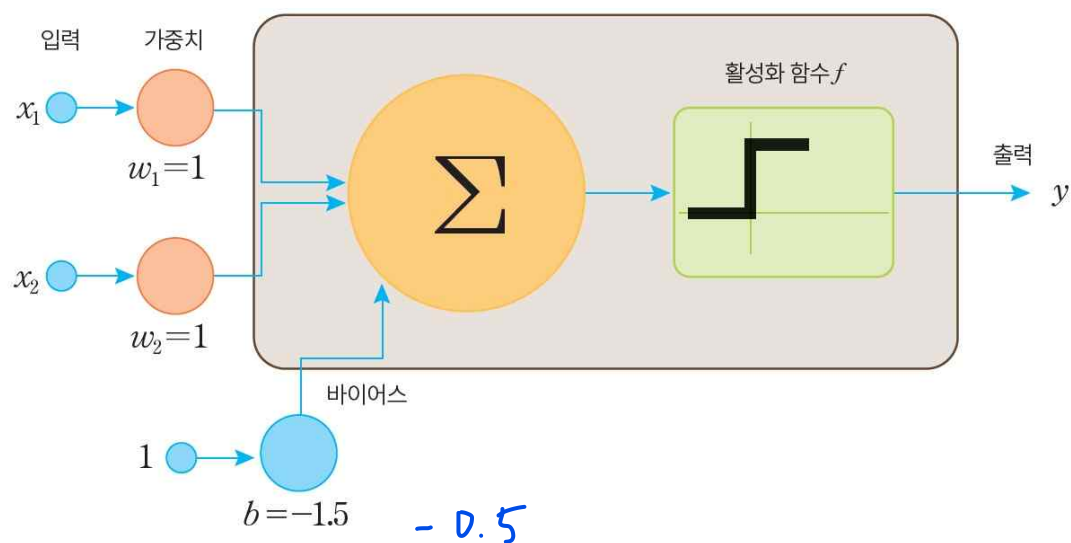
$$y = \begin{cases} 1 & \text{if } (w_1x_1 + w_2x_2 > \theta) \\ 0 & \text{otherwise} \end{cases}$$

여기서 θ 는 임계값이다. 하지만 계산과정에서 불편함을 야기한다. 따라서 연구자들은 임계값 대신에 바이어스(bias, 편향)을 도입하였다. 바이어스 b 는 $-\theta$ 로 정의된다. 따라서 바이어스를 사용하면 퍼셉트론의 식을 다음과 같이 바꿔서 작성할 수 있다.

$$y = \begin{cases} 1 & \text{if } (w_1x_1 + w_2x_2 + b > 0) \\ 0 & \text{otherwise} \end{cases}$$

이 두 개의 식은 기호만 다를 뿐 본질적으로 동일하다. 하지만 바이어스를 정의하면, 임계값을 입력이 항상 1인 연결선의 가중치로 생각할 수 있어서 편리하다.

퍼셉트론은 논리 연산을 학습할 수 있을까?



x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

가중치를 -1.5로 정해주면 and기능을 한

그림 12-7 논리 연산을 하는 퍼셉트론

- 0.5
or

퍼셉트론은 논리 연산을 학습할 수 있을까?

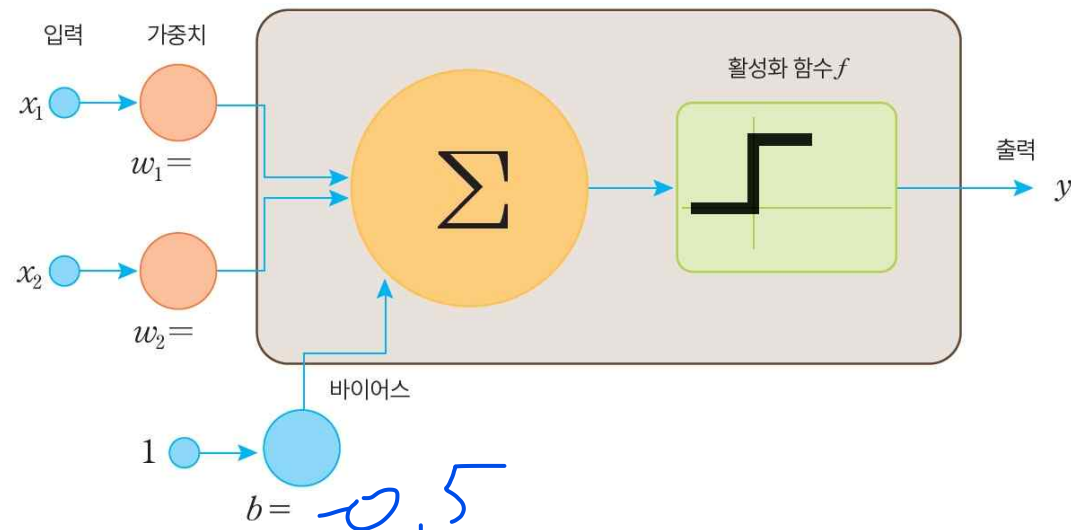
표 12-2 퍼셉트론 출력 계산

x1	x2	$w_1x_1+w_2x_2$	b	$w_1x_1+w_2x_2+b$	출력
0	0	$1*0+1*0=0$	-1.5	-1.5	0
1	0	$1*1+1*0=1$	-1.5	-0.5	0
0	1	$1*0+1*1=1$	-1.5	-0.5	0
1	1	$1*1+1*1=2$	-1.5	+0.5	1

Q

- 퍼셉트론이 **OR** 연산을 학습한 것처럼 동작하려면 가중치와 바이어스는 어떤 값이어야 할까? 표를 만들어서 계산하라.

$$y = \begin{cases} 1 & \text{if } (w_1x_1 + w_2x_2 + b \geq 0) \\ 0 & \text{otherwise} \end{cases}$$



활성화 함수

- 계단 함수

불연속 함수, 미분이 안되는 함수

계단(step) 함수

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

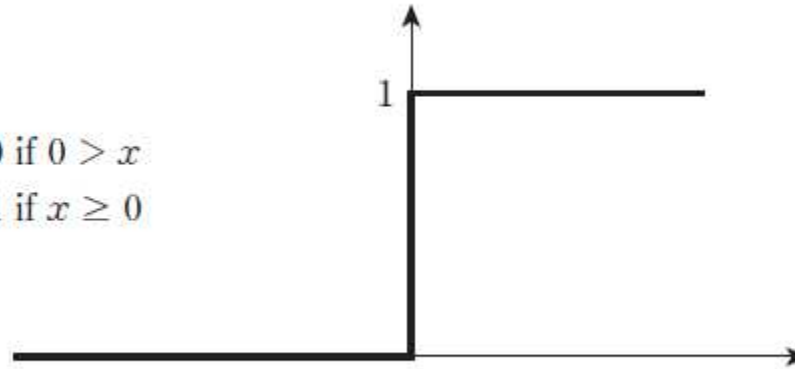


그림 5-7 퍼셉트론에서의 활성화 함수

퍼셉트론 구현 #1(순수 파이썬 사용)

```
epsilon = 0.0000001
```

```
def perceptron(x1, x2):  
    w1, w2, b = 1.0, 1.0, -1.5  
    sum = x1*w1+x2*w2+b  
    if sum > epsilon :  
        return 1  
    else :  
        return 0
```

```
print(perceptron(0, 0))  
print(perceptron(1, 0))  
print(perceptron(0, 1))  
print(perceptron(1, 1))
```

컴퓨터로는 2진수로 실수를 나타내면 오류가 생겨서

부동소수점 오차를 방지하기 위하여

```
0  
0  
0  
1
```


퍼셉트론 구현 #2(넘파이 사용)

```
import numpy as np
epsilon = 0.0000001

def perceptron(x1, x2):
    X = np.array([x1, x2])
    W = np.array([1.0, 1.0])
    B = -1.5
    sum = np.dot(W, X)+B
    if sum > epsilon :
        return 1
    else :
        return 0

print(perceptron(0, 0))
print(perceptron(1, 0))
print(perceptron(0, 1))
print(perceptron(1, 1))
```

```
0
0
0
1
```

03 퍼셉트론 학습 알고리즘

- 학습이라고 부르려면 신경망이 스스로 가중치를 자동으로 설정해주는 알고리즘이 필요하다. 퍼셉트론에서도 학습 알고리즘이 존재한다.

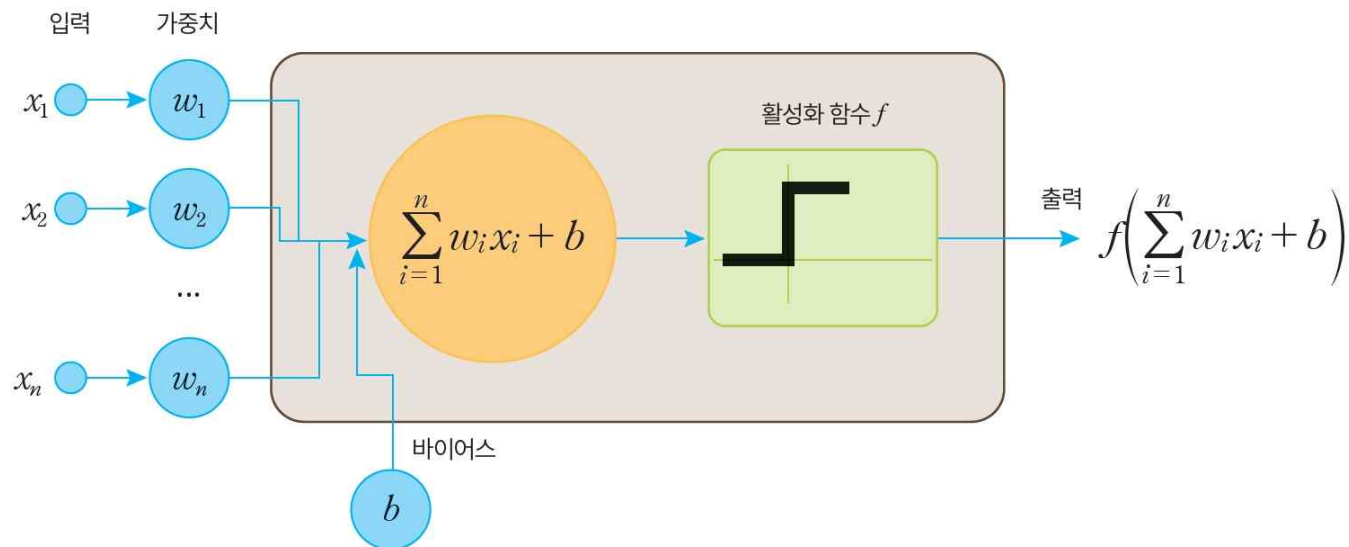


그림 12-8 퍼셉트론

퍼셉트론 학습 알고리즘

input: 학습 데이터 $(x^1, d^1), \dots, (x^m, d^m)$

① 모든 w 들과 바이어스 b 를 0 또는 작은 난수로 초기화한다.

② while (가중치가 변경되지 않을 때까지 반복)

③ 각 학습 데이터 x^k 와 정답 d^k 에 대하여

④ $y^k(t) = f(w \cdot x)$ 현재 상태에 대한 출력

⑤ 모든 가중치 w_i 에 대하여 $w_i(t+1) = w_i(t) + \eta \cdot (d^k - y^k(t)) \cdot x_i^k$

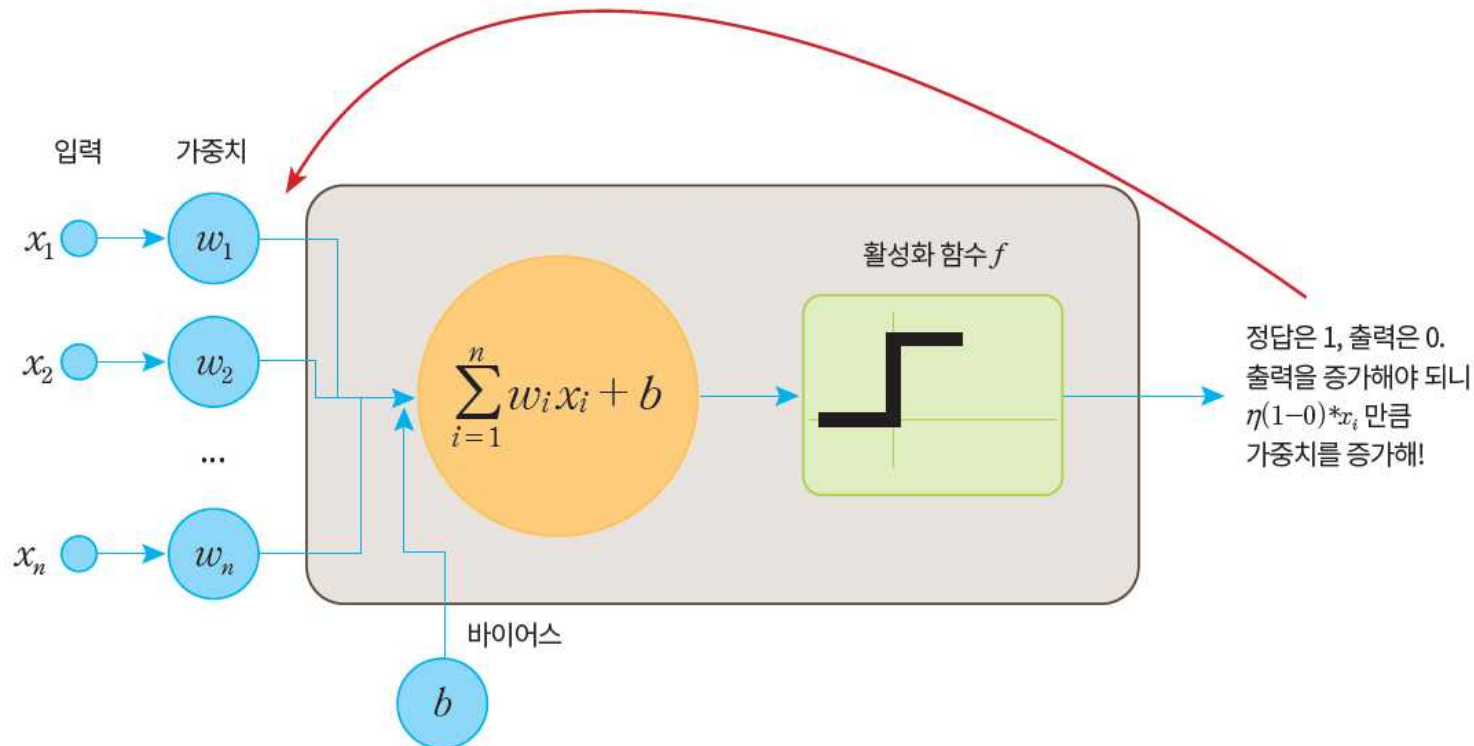
에다가 이만큼을 더해줌

그이전 가중치

→ 입력

정답 - 출력 = 오차

퍼셉트론 학습 알고리즘의 설명



- 퍼셉트론이 1을 0으로 잘못 식별했다고 하자. 가중치의 변화량은 $\eta^* (1-0) \times x_i^k$ 가 된다. 따라서 가중치는 증가된다. 가중치가 증가되면 출력도 증가되어서 출력이 0에서 1이 될 가능성이 있다.
- 반대로 0을 1로 잘못 식별했다고 하자. 가중치의 변화량은 $\eta^* (0-1) \times x_i^k$ 가 된다. 따라서 가중치는 줄어든다. 가중치가 줄어들면 출력도 감소되어서 출력이 1에서 0이 될 가능성이 있다.

논리 연산자 AND 학습 과정

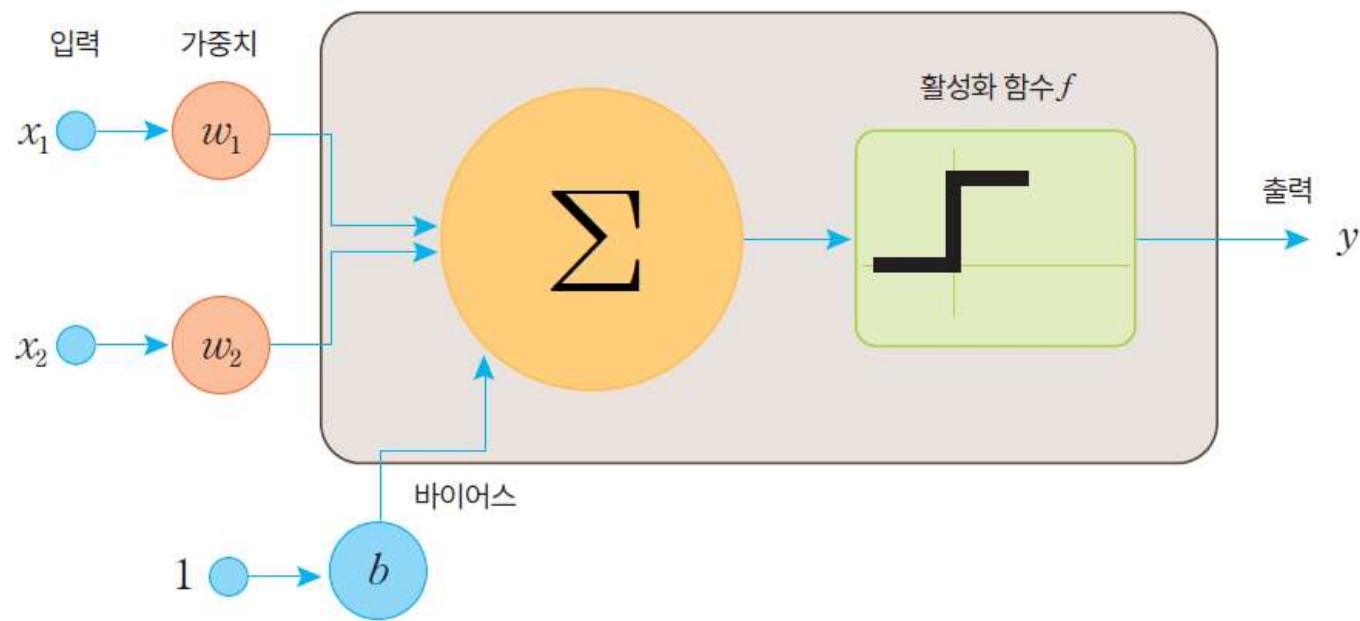


그림 12-11 퍼셉트론

$$w(1) = w(0) + \eta * (d - y) * x(0)$$

$$-0.1 + 0 = -0.1$$

- 퍼셉트론을 이용하여 AND 연산을 학습시키는 과정. $\eta = 0.1$ $w(1) = -0.1$
 - $y = x_1 * w_1 + x_2 * w_2 + 1 * b$, $\eta = 0.1$, $w(t+1) = w(t) + \eta * (d - y) * x(t)$

x0	x1	x2	d	Y	d-y	b	w1	w2	
				<u>> 0</u>		0	0	0	
1	0	0	0	1	-1	<u>-0.1</u>	0	0	
1	1	0	0	0	0	-0.1	0	0	
1	0	1	0	0	0	-0.1	0	0	
1	1	1	1	0	1	0	0.1	0.1	
1	0	0	0	1	-1	-0.1	0.1	0.1	입력이 1,1,1이라 더해주어야 한다.
1	1	0	0	1	-1	-0.2	0	0.1	
1	0	1	0	0	0	-0.2	0	0.1	
1	1	1	1	0	1	-0.1	0.1	0.2	

y는 0보다 크거나 같으면 이다. 첫출력 0 이니까 값

$$-0.1 + 0.1 - 0.1 = -0.1$$

$$w =$$

퍼셉트론 프로그래밍

```
import numpy as np

epsilon = 0.0000001

def step_func(t):    # 퍼셉트론의 활성화 함수
    if t > epsilon: return 1    # 부동소수점 반올림하기
    else: return 0

X = np.array([      # 훈련 데이터 세트
    [0, 0, 1],      # 맨 끝의 1은 바이어스를 위한 입력 신호 1이다.
    [0, 1, 1],      # 맨 끝의 1은 바이어스를 위한 입력 신호 1이다.
    [1, 0, 1],      # 맨 끝의 1은 바이어스를 위한 입력 신호 1이다.
    [1, 1, 1]       # 맨 끝의 1은 바이어스를 위한 입력 신호 1이다.
])
```

퍼셉트론 프로그래밍

```
y = np.array([0, 0, 0, 1]) # 정답을 저장하는 넘파이 행렬
W = np.zeros(len(X[0])) # 가중치를 저장하는 넘파이 행렬

def perceptron_fit(X, Y, epochs=10): # 퍼셉트론 학습 알고리즘 구현
    global W
    eta = 0.2 # 학습률

    for t in range(epochs):
        print("epoch=", t, "=====")
        for i in range(len(X)):
            predict = step_func(np.dot(X[i], W))
            error = Y[i] - predict # 오차 계산
            W += eta * error * X[i] # 가중치 업데이트
            print("현재 처리 입력=", X[i], "정답=", Y[i], "출력=", predict, "변경된 가중치=", W)
        print("=====")

def perceptron_predict(X, Y): # 예측
    global W
    for x in X:
        print(x[0], x[1], "->", step_func(np.dot(x, W)))

perceptron_fit(X, y, 6)
perceptron_predict(X, y)
```


실행결과

epoch= 0 =====

현재 처리 입력= [0 0 1] 정답= 0 출력= 0 변경된 가중치= [0. 0. 0.]

현재 처리 입력= [0 1 1] 정답= 0 출력= 0 변경된 가중치= [0. 0. 0.]

현재 처리 입력= [1 0 1] 정답= 0 출력= 0 변경된 가중치= [0. 0. 0.]

현재 처리 입력= [1 1 1] 정답= 1 출력= 0 변경된 가중치= [0.2 0.2 0.2]

=====

epoch= 1 =====

현재 처리 입력= [0 0 1] 정답= 0 출력= 1 변경된 가중치= [0.2 0.2 0.]

현재 처리 입력= [0 1 1] 정답= 0 출력= 1 변경된 가중치= [0.2 0. -0.2]

현재 처리 입력= [1 0 1] 정답= 0 출력= 0 변경된 가중치= [0.2 0. -0.2]

현재 처리 입력= [1 1 1] 정답= 1 출력= 0 변경된 가중치= [0.4 0.2 0.]

=====

epoch= 2 =====

현재 처리 입력= [0 0 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.2 0.]

현재 처리 입력= [0 1 1] 정답= 0 출력= 1 변경된 가중치= [0.4 0. -0.2]

현재 처리 입력= [1 0 1] 정답= 0 출력= 1 변경된 가중치= [0.2 0. -0.4]

현재 처리 입력= [1 1 1] 정답= 1 출력= 0 변경된 가중치= [0.4 0.2 -0.2]

=====

epoch= 3 =====

현재 처리 입력= [0 0 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.2 -0.2]

현재 처리 입력= [0 1 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.2 -0.2]

현재 처리 입력= [1 0 1] 정답= 0 출력= 1 변경된 가중치= [0.2 0.2 -0.4]

현재 처리 입력= [1 1 1] 정답= 1 출력= 0 변경된 가중치= [0.4 0.4 -0.2]

=====

실행결과

epoch= 4 =====

현재 처리 입력= [0 0 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.4 -0.2]

현재 처리 입력= [0 1 1] 정답= 0 출력= 1 변경된 가중치= [0.4 0.2 -0.4]

현재 처리 입력= [1 0 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.2 -0.4]

현재 처리 입력= [1 1 1] 정답= 1 출력= 1 변경된 가중치= [0.4 0.2 -0.4]

epoch= 5 =====

현재 처리 입력= [0 0 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.2 -0.4]

현재 처리 입력= [0 1 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.2 -0.4]

현재 처리 입력= [1 0 1] 정답= 0 출력= 0 변경된 가중치= [0.4 0.2 -0.4]

현재 처리 입력= [1 1 1] 정답= 1 출력= 1 변경된 가중치= [0.4 0.2 -0.4]

0 0 -> 0

0 1 -> 0

1 0 -> 0

1 1 -> 1

Q

- 이번에는 퍼셉트론을 이용하여 **OR** 연산을 학습시키는 과정을 표로 만들어라. $\eta = 0.1$

$y = x_1 * w_1 + x_2 * w_2 + 1 * b$, $\eta = 0.1$, $w(t+1) = w(t) + \eta * (d - y) * x(t)$

x0	x1	x2	d	y	d-y		b	w1	w2	
							0	0	0	
1	0	0	0	1	-1		-0.1	0	0	
1	1	0	1	0	1		0	0.1	0	
1	0	1	1	1	0		0	0.1	0	
1	1	1	1	1	0		0	0.1	0	
1	0	0	0	1	-1		-0.1	0.1	0	
1	1	0	1	1	0		-0.1	0.1	0	
1	0	1	1	0	1		0	0.1	0.1	
1	1	1	1	1	0		0	0.1	0.1	

04 sklearn으로 퍼셉트론 실습하기

```
from sklearn.linear_model import Perceptron

# 샘플과 레이블이다.
X = [[0,0],[0,1],[1,0],[1,1]]
y = [0, 0, 0, 1]

# 퍼셉트론을 생성한다. tol는 종료 조건이다. random_state는 난수의 시드이다.
clf = Perceptron(tol=1e-3, random_state=0)

# 학습을 수행한다.
clf.fit(X, y)

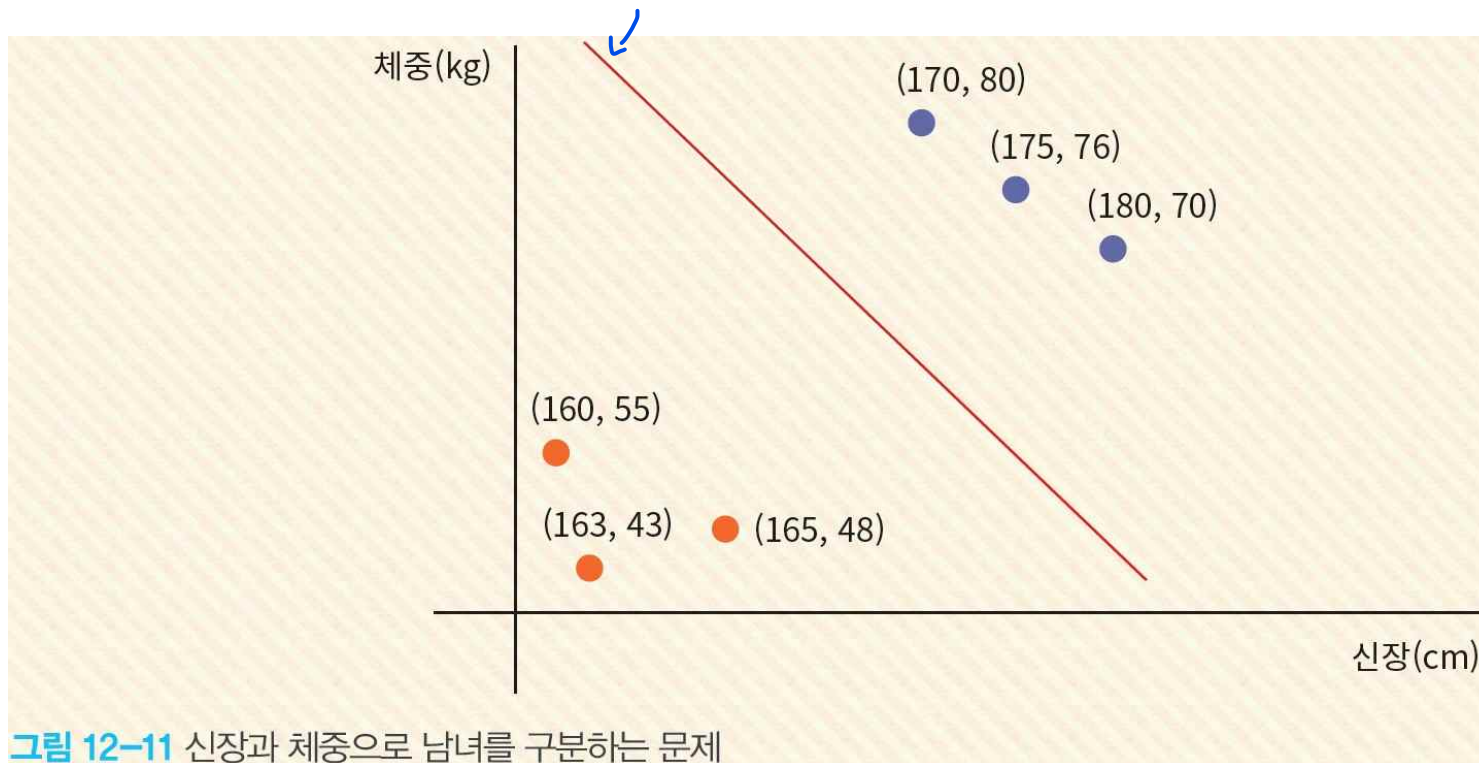
# 테스트를 수행한다.
print(clf.predict(X))
```

```
[0 0 0 1]
```

Lab: 퍼셉트론으로 분류

- 대학생들의 **신장**과 **체중**을 받아서 성별을 출력하는 퍼셉트론을 만들어보자.

빨간선을 찾아주는 게 퍼셉트론이 하는 일



Q

- 앞장에서 아이리스 데이터를 분류해본 바 있다. 퍼셉트론으로 이것을 시도해보라. 다음 코드를 참조한다

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron

iris = load_iris()
# 입력과 출력을 설정한다.
X = iris.data[:, (0, 1)] # 꽃받침의 너비와 높이만을 입력으로 한다
y = (iris.target == 0).astype(int) # 출력은 'Iris Setosa 인가 아닌가'이다

percep = Perceptron(random_state=32) # random 시작 값을 고정함
percep.fit(X, y)
print(percep.score(X, y))
```

퍼셉트론의 한계점

- XOR 연산

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

그림 12-12 논리적인 XOR 연산

XOR 학습 문제

```
...  
# XOR 연산 학습 데이터셋  
# 샘플과 레이블이다.  
X = [[0,0],[0,1],[1,0],[1,1]]  
y = [0, 1, 1, 0]  
  
weights = [0.0, 0.0]  
bias = 0.0  
  
l_rate = 0.1                # 학습률  
n_epoch = 100               # 에포크 횟수  
weights = train_weights(X, y, l_rate, n_epoch)  
print(weights, bias)
```


실행결과

```
...
에포크 번호=97, 학습률=0.100, 오류=4.000
[-0.1, 0.0] -0.1
[-0.1, 0.1] 0.0
[0.0, 0.1] 0.1
[-0.1, 0.0] 0.0
에포크 번호=98, 학습률=0.100, 오류=4.000
[-0.1, 0.0] -0.1
[-0.1, 0.1] 0.0
[0.0, 0.1] 0.1
[-0.1, 0.0] 0.0
에포크 번호=99, 학습률=0.100, 오류=4.000
```

XOR 학습 문제

```
from sklearn.linear_model import Perceptron

# XOR 학습 예제와 정답 레이블이다.
X = [[0,0],[0,1],[1,0],[1,1]]
y = [0, 1, 1, 0]

# 퍼셉트론을 생성한다. tol는 종료 조건이다. random_state는 난수의 시드이다.
clf = Perceptron(tol=1e-3, random_state=0, verbose=1)

# 학습을 수행한다.
clf.fit(X, y)

# 테스트를 수행한다.
print(clf.predict(X))
```

실행결과

- 학습이 전혀 이루어지지 않는다. 퍼셉트론은 **AND**나 **OR** 연산은 학습할 수 있는데, 왜 **XOR** 연산은 학습할 수 없을까?

[0 0 0 0]

퍼셉트론은 기능이 선형 분류이다.
2차원에서 직선을 그리는 일 밖에 못한다.

선형 분류 가능 문제

- 패턴 인식 측면에서 보면 퍼셉트론은 직선을 이용하여 입력 패턴을 분류하는 선형 분류자(linear classifier)의 일종이라고 말할 수 있다.

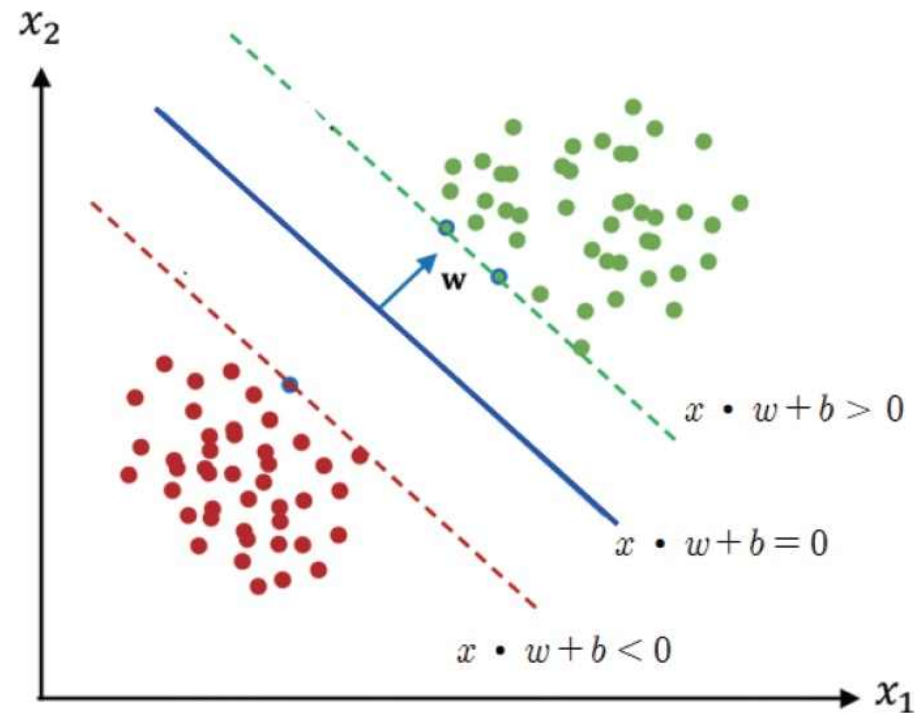
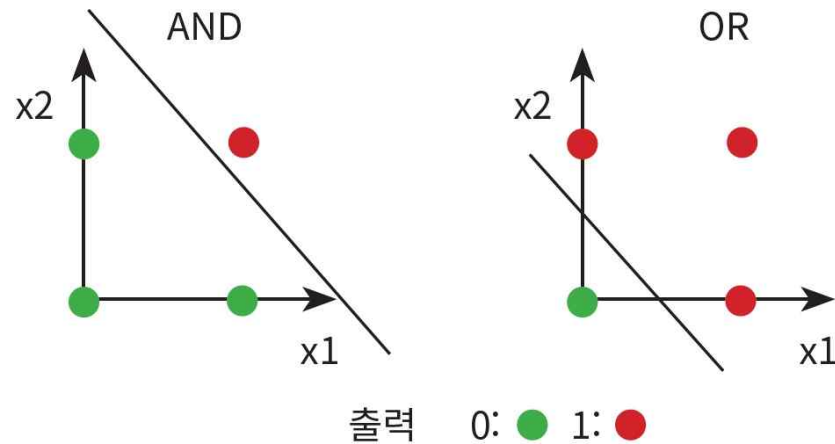


그림 12-12 선형 분류자

06 선형 분류 가능 문제

- Minsky와 Papert는 1969년에 발간된 책 “Perceptrons”에서 1개의 레이어(layer, 계층)로 구성된 퍼셉트론은 XOR 문제를 학습할 수 없다는 것을 수학적으로 증명



XOR 학습 문제



그림 12-14 선형 분리 불가능한 문제

다층 퍼셉트론으로 XOR 문제를 해결

- XOR 연산은 하나의 직선으로는 올바르게 분류할 수 없다. 하지만 2개의 직선을 사용하면 어떨까?
- 2개의 직선을 사용한다면 XOR 입력을 분류할 수 있다.

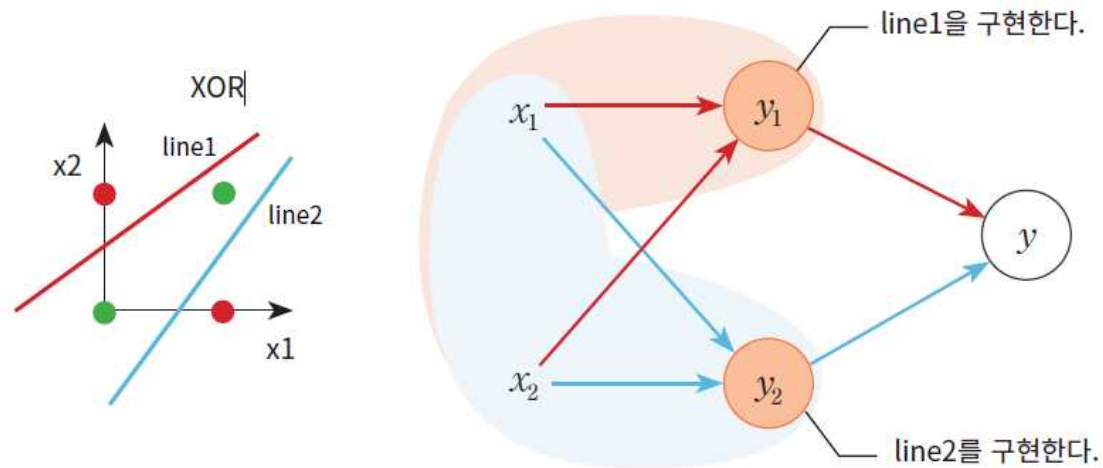


그림 12-16 다층을 사용하는 퍼셉트론

하는법은 예전에도 알았는데 왜 안됐을까.

다층 퍼셉트론으로 XOR 문제를 해결

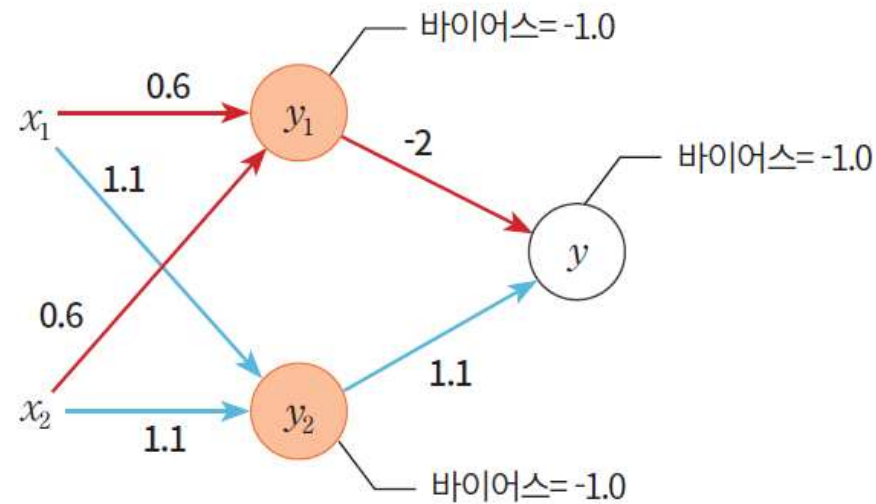
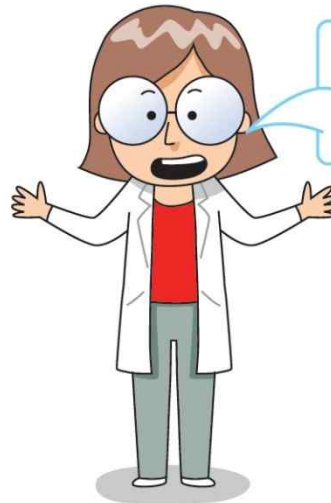
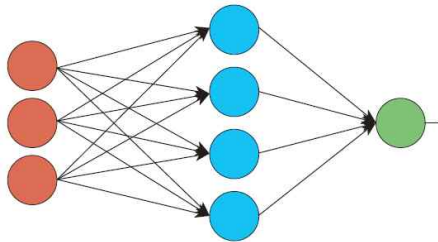


그림 12-17 다층 퍼셉트론에서 XOR 문제 해결

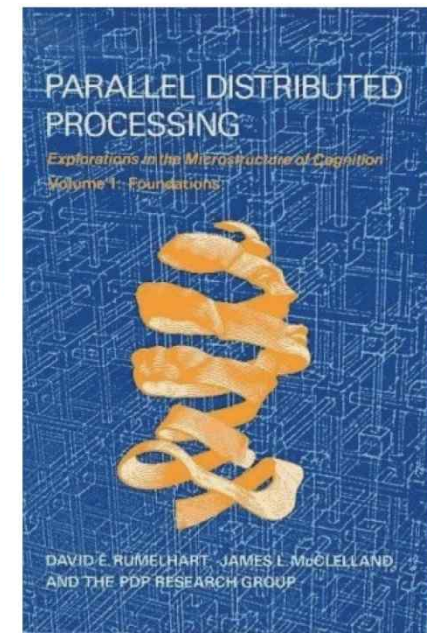
x1	x2	y1	y2	y	출력
0	0	0	1	0	0
1	0	1	1	1	1
0	1	1	1	1	1
1	1	1	0	0	0

다층 퍼셉트론의 학습 알고리즘

- Minsky와 Papert는 다층 퍼셉트론을 학습시키는 알고리즘을 찾기가 아주 어려울 것이라고 예언하였다. 예언에 맞았을까? 그렇지 않았다. 1980년대 중반에 Rumelhart와 Hinton 등은 다층 퍼셉트론을 위한 학습 알고리즘을 재발견하게 된다.



이것이 바로 역전파
알고리즘이 실렸던 책이다.



Summary

- 딥러닝(deep learning)의 시작은 1950년대부터 연구되어 온 인공 신경망(artificial neural network: ANN)이다.
- 신경망의 가장 큰 장점은 학습이 가능하다는 점이다. 데이터만 주어지면 신경망은 예제로부터 배울 수 있다.
- 뉴런은 다른 뉴런들로부터 신호를 받아서 모두 합한 후에 비선형 함수를 적용하여 출력을 계산한다. 연결선은 가중치를 가지고 있고 이 가중치에 학습의 결과가 저장된다.
- 퍼셉트론은 하나의 뉴런만을 사용한다. 다수의 입력을 받아서 하나의 신호를 출력하는 장치이다
- 퍼셉트론은 AND나 OR 같은 논리적인 연산을 학습할 수 있었지만 XOR 연산은 학습할 수 없었다. 선형 분리 가능한 문제만 학습할 수 있었다.

Q & A

